

Build a Graph Database Application

Build an application backed by a graph database, using CognoDB as the database layer.

Deliverable: A GitHub repository URL (hosted demo link optional but encouraged)

Deadline: 48 hours from receiving this assignment

Submit to: hr@wexa.ai

1. Technology stack

This assignment uses CognoDB, a managed graph database, as the data layer for your application. CognoDB speaks openCypher over the Bolt protocol (Bolt 5.0–5.4) and works with the official Neo4j drivers for Python, JavaScript, Go, Java and .NET, so you can use your language's standard driver rather than a custom SDK. CognoDB Cloud (console.cognodb.com) provides a database instance in under a minute, and the free tier requires no credit card.

2. Objective

Build a small, complete application backed by a graph database. The application idea is entirely up to you. Wexa has no specific use case in mind and no stake in which one you pick. This assignment is meant to show how you approach data modeling, engineering architecture, and (if you choose to use them) AI-assisted coding tools. Pick an idea you're genuinely interested in. Our objective is to assess whether you can apply graph data modeling to build a working application.

3. Set up CognoDB Cloud

- 1. Create an account.** Go to <https://console.cognodb.com/signup> and sign up. The free tier requires no credit card.
- 2. Create a free instance.** From the console, create a free (c0) instance and pick a region. It provisions in under a minute. Each workspace gets one free instance.
- 3. Save your connection details.** You will get a connection URI of the form `bolt+s://<instance-id>.databases.cognodb.cloud` and a generated password for the user "cognodb". The password is shown exactly once — copy or download it immediately and store it where your code reads its secrets.
- 4. Connect with an official Neo4j driver.** Install the official Neo4j driver for your language, point it at your `bolt+s://` URI with username "cognodb" and your saved password, and run your first Cypher query. No other code changes are needed.

Free tier limits

The free (c0) instance is small: burstable 0.5 vCPU, 256 MB RAM, 1 GB disk, up to 200 connections. Size your dataset accordingly — a few thousand to a few hundred thousand nodes and relationships is enough to demonstrate your use case clearly.

4. Choose your use case

The use case is entirely your choice. Pick any real-world problem where the interesting questions are about connections and relationships rather than rows in a table. Part of what we are evaluating is your judgment in choosing a problem where a graph database genuinely earns its place, so choose something you can argue for convincingly. Originality counts.

Whatever you choose, the README must include a short "Why a graph database?" section explaining what your use case gains over a relational schema.

5. Requirements

5.1 Data & queries

- A thoughtful graph data model: labeled nodes, typed relationships and properties, documented with a simple diagram in the README.
- Real or realistic seed data, loaded by a script included in the repo.
- Cypher queries that exercise the graph including at least one multi-hop traversal (2 hops or more) and at least one query a relational database would find awkward.
- Parameterised queries via the official Neo4j driver. No string-concatenated Cypher.

5.2 Application & UI/UX

- A functional web application (any stack you like) that a non-technical person could use to explore the use case.
- Clean, intentional UI and UX: sensible layout and navigation, loading and empty states, readable typography. Design effort is explicitly part of the evaluation.

5.3 Engineering

- Connection details (URI, password) read from environment variables never committed to the repository.
- Clear project structure and a codebase you would be comfortable walking us through line by line.
- Graceful error handling when the database is unreachable.

6. Deliverables

A GitHub repository containing:

- **Full source code** : application, data-loading scripts and Cypher queries.
- **README** : the use case and "Why a graph database?", a data model diagram, setup and run instructions (including how to create the CognoDB instance), the main queries explained, and screenshots of the UI.
- **Mandatory** : a hosted application demo link (any free hosting tier) and a short screen recording.

If you wish to keep the github repo private, please provide us access to the repo.

7. What a strong submission looks like

At minimum, deliver a working app with a sound data model, a seed script, and a working application that solves a real-world problem of your choosing. Beyond that, submissions stand out through:

- Polished UX: thoughtful interactions and proper loading, empty and error states.
- Well-structured architecture: configuration, error handling, sensible layering.

- A hosted demo, a clear end-to-end use case, and code others could maintain.

8. Submission

- A GitHub repository.
- Email the repository URL (and demo link, if any) to hr@wexa.ai with the subject line “CognoDB Assignment 2 – <Your Name>”.
- Submit within 48 hours of receiving this assignment.
- Keep your instance running until you hear back from us, in case we need to try the app against live data.

9. Questions & support

If you hit a problem with CognoDB Cloud itself (signup, provisioning, connectivity), email cognodb@wexa.ai. For questions about the assignment, reply to the email that delivered it. Use of AI coding assistants is fine — you must be able to explain and defend every part of your submission in the follow-up interview.